Data Structures BS (CS) _Fall_2025

Lab_11 Manual

Learning Objectives:

1. Understand the concept and structure of AVL Trees (self-balancing Binary Search Trees).
2. Implement AVL Tree operations using linked nodes.
3. Learn how to maintain balance through rotations.
4. Analyze the time complexities of AVL Tree operations.

AVL Trees

An AVL Tree is a self-balancing Binary Search Tree where the height difference (or balance factor) between the left and right subtrees of any node is at most 1.

For every node:

BalanceFactor = height(leftSubtree) - height(rightSubtree)

- If the balance factor is -1, 0, or +1, the tree is balanced.
- If the balance factor is less than -1 or greater than +1, the tree becomes unbalanced and requires rotation to restore balance.

This ensures AVL Trees always remain approximately balanced, leading to efficient insertion, deletion, and search operations.

Node Structure

Each node in an AVL Tree contains:

• data → key or value

• left → pointer to left child

• right → pointer to right child

• height → height of node used to calculate balance factor

```
struct Node {
   int data;
   Node* left;
   Node* right;
```

```
    int height;

    Node(int val) {
        data = val;
        left = right = nullptr;
        height = 1;
    }
};
```

Helper Functions

```
int getHeight(Node* node) {
    return (node == nullptr) ? 0 : node->height;
}

int getBalance(Node* node) {
    return (node == nullptr) ? 0 : getHeight(node->left) - getHeight(node->right);
}

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

Rotations

Rotations are used to restore balance when the tree becomes unbalanced after insertion or deletion.

Right Rotation (LL Rotation)

```
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;

    return x;
}
```

Left Rotation (RR Rotation)

```
Node* leftRotate(Node* x) {
   Node* y = x->right;
   Node* T2 = y->left;

   y->left = x;
   x->right = T2;

   x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
   y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

   return y;
}
```

Left-Right Rotation (LR Rotation)

```
Node* leftRightRotate(Node* node) {
   node->left = leftRotate(node->left);
   return rightRotate(node);
}
```

Right-Left Rotation (RL Rotation)

```
Node* rightLeftRotate(Node* node) {
   node->right = rightRotate(node->right);
   return leftRotate(node);
}
```

Insertion in AVL Tree

```
Node* insert(Node* root, int key) {
   if (root == nullptr)
      return new Node(key);

   if (key < root->data)
      root->left = insert(root->left, key);
   else if (key > root->data)
      root->right = insert(root->right, key);
   else
      return root; // Duplicate keys not allowed

   root->height = 1 + max(getHeight(root->left), getHeight(root->right));
   int balance = getBalance(root);
```

```
    if (balance > 1 && key < root->left->data)
        return rightRotate(root);  // LL
    if (balance < -1 && key > root->right->data)
        return leftRotate(root);   // RR
    if (balance > 1 && key > root->left->data)
        return leftRightRotate(root);  // LR
    if (balance < -1 && key < root->right->data)
        return rightLeftRotate(root);  // RL

    return root;
}
```

Deletion in AVL Tree

```
Node* findMin(Node* node) {
    Node* current = node;
    while (current->left != nullptr)
        current = current->left;
    return current;
}

Node* deleteNode(Node* root, int key) {
    if (root == nullptr)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if ((root->left == nullptr) || (root->right == nullptr)) {
            Node* temp = root->left ? root->left : root->right;
            if (temp == nullptr) {
                temp = root;
                root = nullptr;
            } else
                *root = *temp;
            delete temp;
        } else {
            Node* temp = findMin(root->right);
            root->data = temp->data;
```

```
            root->right = deleteNode(root->right, temp->data);
        }
    }

    if (root == nullptr)
        return root;

    root->height = 1 + max(getHeight(root->left), getHeight(root->right));
    int balance = getBalance(root);

    if (balance > 1 && getBalance(root->left) >= 0)
        return rightRotate(root);
    if (balance > 1 && getBalance(root->left) < 0)
        return leftRightRotate(root);
    if (balance < -1 && getBalance(root->right) <= 0)
        return leftRotate(root);
    if (balance < -1 && getBalance(root->right) > 0)
        return rightLeftRotate(root);

    return root;
}
```

Traversals

```
void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void preorder(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {
    if (root == nullptr) return;
    postorder(root->left);
    postorder(root->right);
```

```
    cout << root->data << " ";
}
```

Time Complexities

Operation	Best Case	Average Case	Worst Case	Description
Insert	O(log n)	O(log n)	O(log n)	Balanced insertion
Search	O(log n)	O(log n)	O(log n)	Always balanced
Delete	O(log n)	O(log n)	O(log n)	Rebalancing keeps height minimal
Traversal	O(n)	O(n)	O(n)	Visit all nodes

Learning Outcome

- Construct and manipulate AVL Trees.
- Implement rotations to maintain balance.
- Perform insertion and deletion with automatic balancing.
- Analyze how self-balancing improves performance compared to regular BSTs.